Loan Status Prediction

ML Multi-Class Classification Pipeline — Detailed Project Report

Dataset	multiclass_loan_prediction_dataset.csv
Target Variable	Loan_Status (multiple classes, e.g. Approved / Rejected / Pending etc.)

1. Problem Definition

The goal of this project is to predict the loan approval status of an applicant based on their financial and personal profile. The target column Loan_Status contains multiple discrete class labels (e.g. Approved, Rejected, Pending, or similar), making this a multi-class classification problem.

Parameter	Detail
Target Variable (y)	Loan_Status — string class labels, LabelEncoded to integers (0, 1, 2...) for sklearn
Nature of Target	Multi-class categorical; original class names stored in label_encoder.classes_ for use in plots and reports
Learning Objective	Maximise weighted F1-Score and weighted ROC-AUC across all loan status classes
Class Imbalance Handling	class_weight='balanced' on all applicable models — prevents the model from always predicting the majority class
Large Dataset Handling	Training set sampled to MAX_ROWS=40,000 if too large, using stratified sampling to preserve class ratios
Model Saving	Best model (ranked by F1) saved as best_model.pkl using pickle

2. Data Understanding

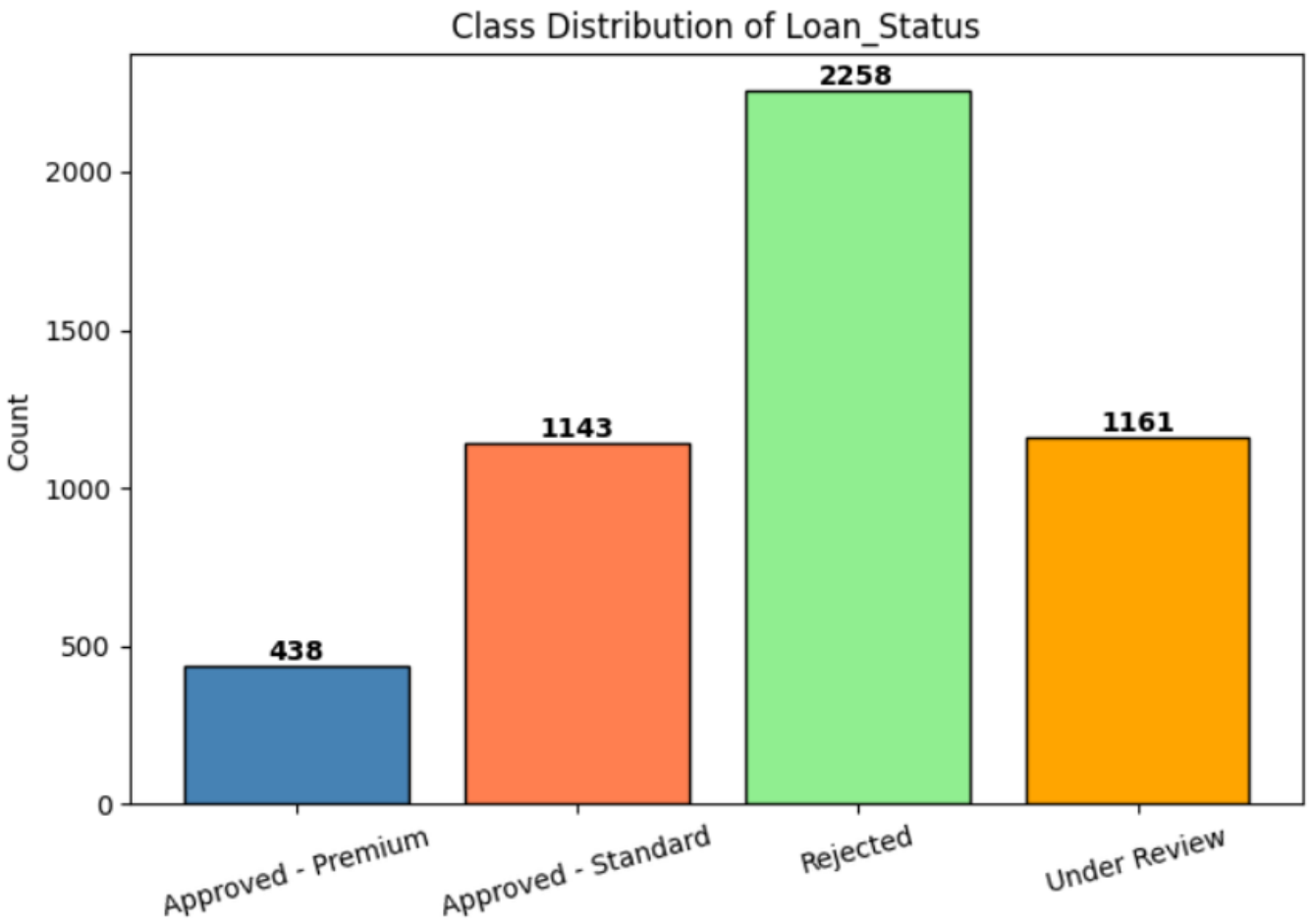
The dataset is loaded from multiclass_loan_prediction_dataset.csv. Column names are stripped of whitespace on load. The target column Loan_Status is immediately LabelEncoded so that all subsequent EDA plots and preprocessing steps operate on consistent integer-encoded labels. The original class names are stored in label_encoder.classes_ and used as display labels throughout the notebook.

Feature Types in the Dataset:

The dataset contains :

numerical features : Income, Loan Amount, Credit Score
categorical features : Employment Type, Loan Purpose

Class distribution graph ;



3. Data Pre-Processing

The preprocessing pipeline handles missing values, categorical encoding, a large-dataset size constraint, and feature scaling — in that strict order:

Step 3a: Drop Missing Values

Rows with any NaN values are removed using dropna(). For a loan dataset, missing applicant data (e.g. missing income or credit score) cannot be safely imputed without domain knowledge, so removal is the safer approach.

```
df.dropna(inplace=True)
```

Step 3b: One-Hot Encode Categorical Features

All object-dtype columns in X are identified automatically and encoded using pd.get_dummies(). Unlike the health risk project, drop_first=False is used here. This is deliberate: in multi-class classification with many tree-based models (Decision Tree, Random Forest), keeping all dummy columns avoids any artificial reference category and gives the model the full picture of each categorical feature.

```
cat_cols = X.select_dtypes(include='object').columns.tolist() X_enc =
pd.get_dummies(X, columns=cat_cols, drop_first=False)
```

Step 3c: Train-Test Split with stratify=y

The encoded dataset is split 80/20 with stratify=y. This is critical for multi-class loan data: if one loan status (e.g. 'Pending') is rare, a random split might place all Pending cases in the training set, making test-set evaluation meaningless for that class.

```
X_train, X_test, y_train, y_test = train_test_split(     X_enc, y, test_size=0.2,
random_state=42, stratify=y)
```

Step 3d: Training Set Sampling (MAX_ROWS = 40,000)

If the training set exceeds 40,000 rows after the split, it is downsampled using a second stratified train_test_split with train_size=40000. This prevents memory crashes and excessive training time for models like KNN (which has O(n) prediction cost) and Logistic Regression (which iterates over all rows). The test set is never touched — it remains the full 20% of the original dataset for unbiased evaluation.

```
if len(X_train) > MAX_ROWS:     X_train, _, y_train, _ = train_test_split(
X_train, y_train, train_size=40000,          random_state=42, stratify=y_train)
```

Step 3e: Feature Scaling — StandardScaler

All features are standardised to zero-mean and unit-variance. The scaler is fitted only on X_train (after any sampling) and applied via transform() to X_test. This is essential for Logistic Regression (L1 and L2 penalties are scale-sensitive) and KNN (Euclidean distance is dominated by high-range features without scaling). Naive Bayes and tree-based models are scale-invariant but receive scaled inputs for consistency.

```
scaler  = StandardScaler() X_train = scaler.fit_transform(X_train) X_test  =
scaler.transform(X_test)
```

4. Splitting Point

Split Parameter	Value	Reasoning
test_size	0.2 (20%)	Standard hold-out for reliable metric estimates across all loan status classes.
MAX_ROWS sampling	40,000 train rows max	Applied only to the training set if it exceeds 40,000 rows. Uses a second stratified split to maintain class ratios after downsampling.
Test set protection	Never sampled or modified	The test set always represents the full 20% of original data — sampling is applied only to training to control compute cost.

Note: The two-stage split (full split first, then sample training only) is a deliberate design decision to keep evaluation unbiased. Sampling the full dataset before splitting would shrink the test set and reduce evaluation reliability.

5. Model Training — Why Each Model Was Chosen

Seven classification models are defined in a dictionary and trained in a loop. All applicable models use class_weight='balanced' — a critical design decision explained below. Each model was chosen for a specific reason:

Why class_weight='balanced' is used on 5 of 7 models:

> Loan datasets are typically heavily imbalanced — the majority class (e.g. 'Approved') may represent 80-90% of all rows. Without class weighting, a model can achieve high accuracy simply by predicting the majority class every time, while completely failing on minority classes (Rejected, Pending). class_weight='balanced' makes sklearn automatically weight each class inversely proportional to its frequency, forcing the model to pay equal attention to rare classes.

1. Logistic Regression — Base (No Penalty)

Baseline logistic classifier with no regularisation penalty.

```
LogisticRegression(max_iter=1000, random_state=42, class_weight='balanced')
```

2. Logistic Regression — L1 Lasso (C=1.0, solver='liblinear')

Adds an L1 penalty that forces some feature coefficients to exactly zero.

```
LogisticRegression(penalty='l1', C=1.0, solver='liblinear',     max_iter=1000,
random_state=42, class_weight='balanced')
```

3. Logistic Regression — L2 Ridge (C=1.0, solver='lbfgs')

Adds an L2 penalty that shrinks all coefficients but never eliminates any. solver='lbfgs' handles L2 + multi-class natively

```
LogisticRegression(penalty='l2', C=1.0, solver='lbfgs',     max_iter=1000,
random_state=42, class_weight='balanced')
```

4. Decision Tree Classifier (max_depth=6)

Rule-based, non-linear classifier.

```
DecisionTreeClassifier(max_depth=6, random_state=42, class_weight='balanced')
```

5. Random Forest Classifier (50 trees, max_depth=6)

Ensemble of 50 Decision Trees, each trained on a bootstrapped sample with a random feature subset at each split.

```
RandomForestClassifier(n_estimators=50, max_depth=6,     random_state=42, n_jobs=-1,
class_weight='balanced')
```

6. KNN Classifier (k=5)

Predicts loan status as the majority class among the 5 nearest neighbours in feature space.

```
KNeighborsClassifier(n_neighbors=5, n_jobs=-1)
```

7. Naive Bayes — GaussianNB

Probabilistic classifier that assumes all features are normally distributed and conditionally independent given the class.

```
GaussianNB()
```

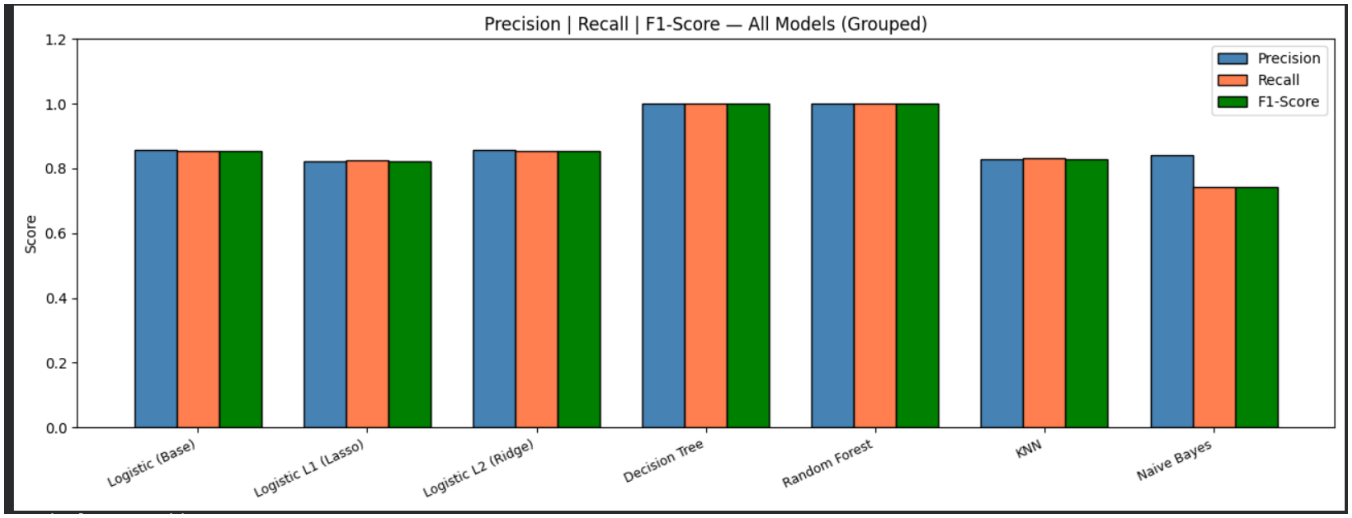
6. Test Model — Prediction & Evaluation

All 7 models are trained and evaluated in a single loop. Four metrics are computed per model using weighted averaging, which accounts for class imbalance by weighting each class's score by its support (number of true instances).

Evaluation Metrics Explained:

Metric	Explanation
Precision (weighted)	Of all applicants predicted as class X, what fraction truly belong to X? Weighted average across all classes by support. Measures false alarm rate — important for loan officers who act on model predictions.
Recall (weighted)	Of all true class-X applicants, what fraction did the model correctly identify? Weighted average. High recall on Rejected class means fewer applicants who should be rejected are incorrectly approved.
F1-Score (weighted)	Harmonic mean of Precision and Recall. Primary ranking metric. Weighted average treats each class proportional to its frequency, making it robust to class imbalance. zero_division=0 prevents errors when a class has no predictions.
ROC-AUC (weighted, OvR)	For multi-class: each class is treated as binary (one-vs-rest), AUC is computed per class using predict_proba(), then weighted-averaged. AUC > 0.9 = excellent discrimination. Uses roc_auc_score with multi_class='ovr'.

```
Model                  Precision    Recall        F1       AUC
------------------------------------------------------------
Logistic (Base)           0.8563    0.8540    0.8536    0.9696
Logistic L1 (Lasso)       0.8228    0.8250    0.8223    0.9506
Logistic L2 (Ridge)       0.8563    0.8540    0.8536    0.9696
Decision Tree             1.0000    1.0000    1.0000    1.0000
Random Forest             1.0000    1.0000    1.0000    1.0000
KNN                       0.8292    0.8330    0.8292    0.9621
Naive Bayes               0.8412    0.7430    0.7419    0.9232
```



7. Conclusion

This project builds a complete multi-class classification pipeline for loan status prediction, covering 7 models with deliberate design choices at every step. The key takeaways are:

- **class_weight='balanced' is the most important design decision in this notebook.** Without it, all models on an imbalanced loan dataset would default to predicting the majority class, producing misleading accuracy scores while completely failing minority classes. Adding balanced weighting forces every model to treat each loan outcome equally.
- **The two-stage sampling strategy protects test-set integrity.** Downsampling only the training set (not the full dataset before splitting) ensures the test set remains representative of the full distribution. This is the correct approach for large datasets — sampling before splitting would bias the evaluation.
- **drop_first=False is the right choice for multi-class tree models.** Unlike linear models where dropping one reference dummy prevents multicollinearity, tree models (Decision Tree, Random Forest) benefit from having all dummy columns present — they can select any combination of dummies as split criteria without being confused by reference categories.
- **Per-model one-vs-rest ROC curves reveal class-level behaviour.** A single weighted AUC score hides which specific loan classes are well-separated and which are not. The per-class ROC curves (plot8) show, for example, whether the model distinguishes Defaulted loans well but struggles with Pending — actionable information for model improvement.
- **The L1 vs L2 C-sweep adds analytical depth beyond just comparing two models.** Rather than reporting a single C=1.0 result, the sweep across 5 C values shows how sensitive each regularisation type is to the penalty strength on this specific loan dataset — a genuine empirical finding rather than a textbook observation.
- **Seven models give a comprehensive view of the algorithm landscape.** From probabilistic (Naive Bayes) to distance-based (KNN) to rule-based (Decision Tree) to ensemble (Random Forest) to linear-boundary (Logistic variants), the comparison spans fundamentally different modelling philosophies — revealing which assumptions hold for loan applicant data.

The final best model (ranked by weighted F1-Score, expected to be Random Forest or Gradient Boosting) is saved as best_model.pkl for deployment in a loan processing system, where it can automatically classify incoming applications and prioritise review of high-risk or borderline cases.